

Foodgram + AI Meal Planner

Общая информация

Название проекта: Foodgram Microservices + AI Meal Planner

Команда: 3 человека

Срок выполнения: 3-4 недели

Формат: Headless REST API с микросервисной архитектурой

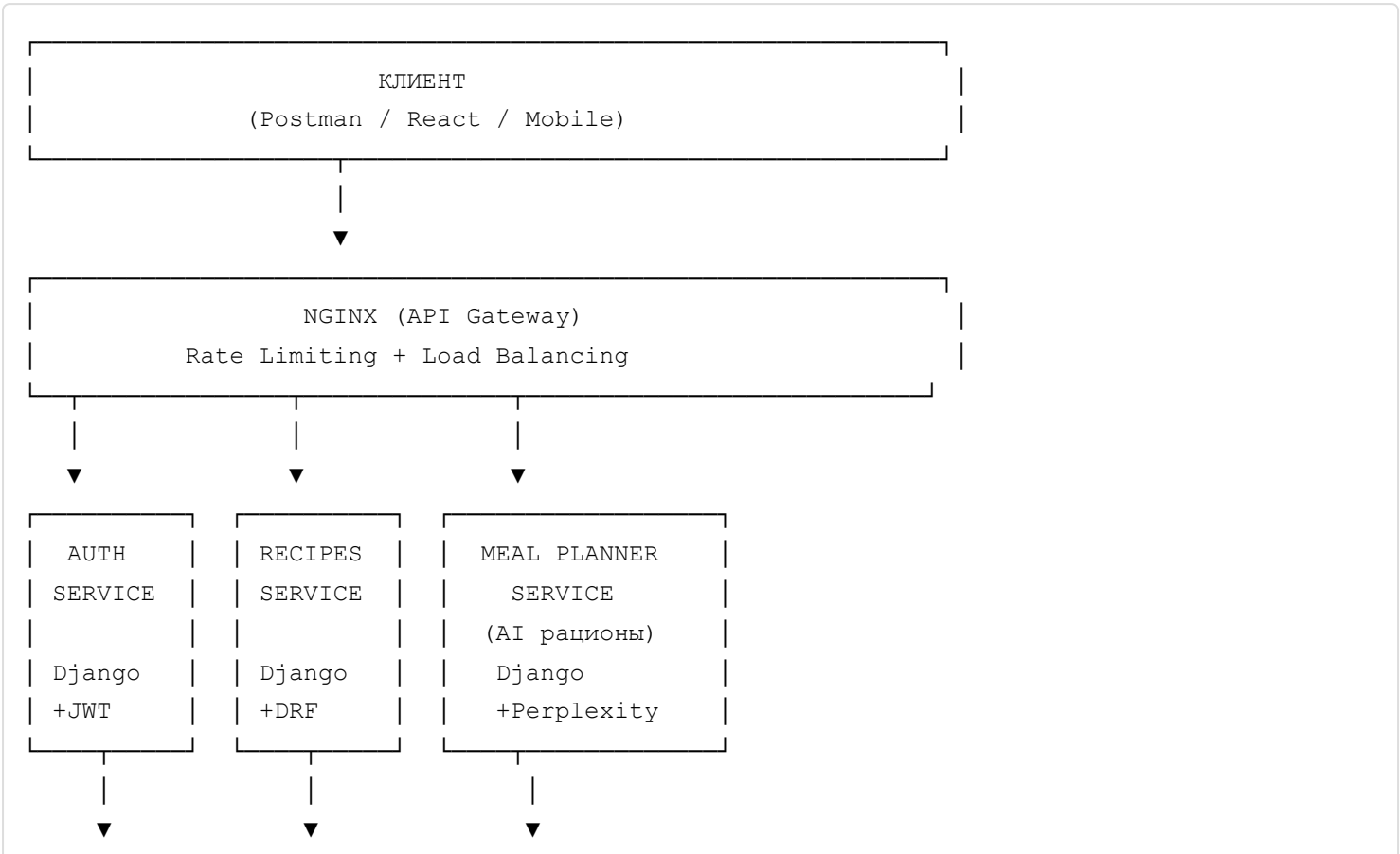
Стек: Python, Django, DRF, PostgreSQL, Redis, Docker, Nginx

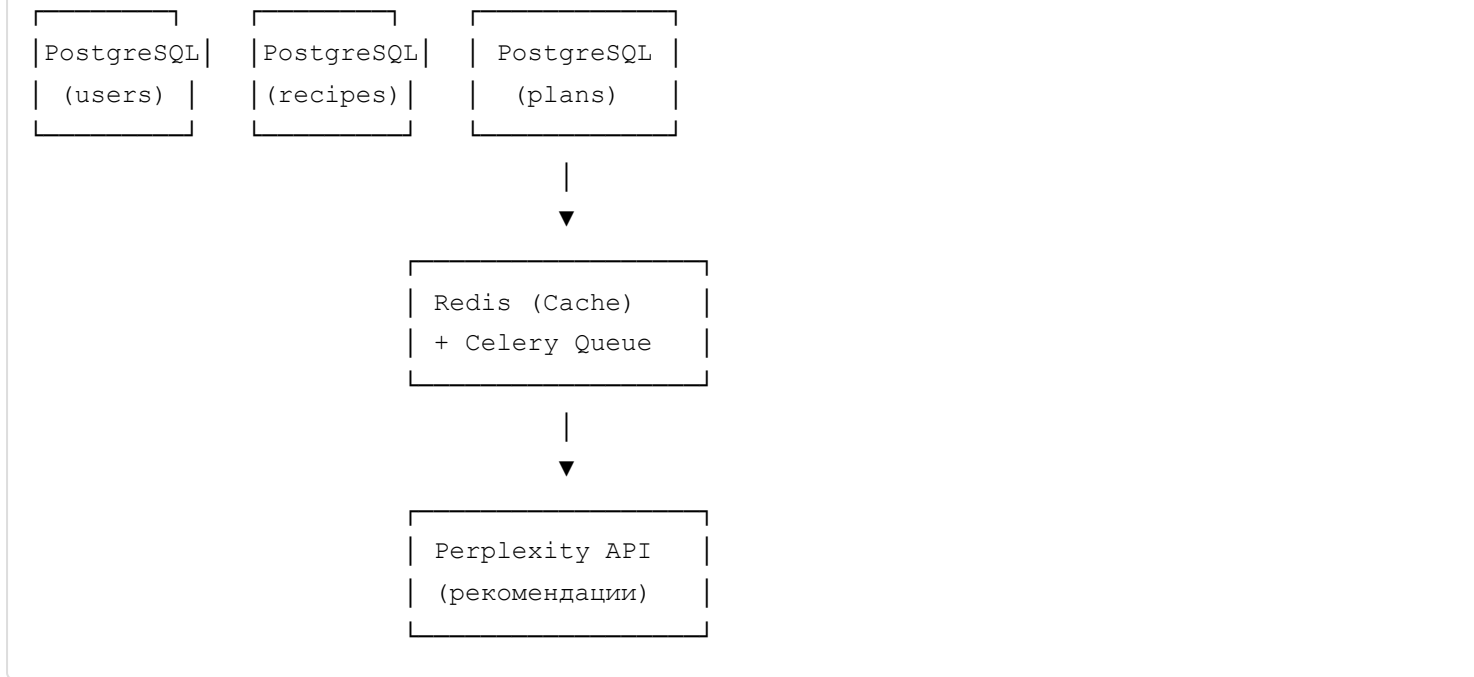
Цель проекта

Создать сервис рецептов с возможностью:

- Публикации и поиска рецептов
- Добавления в избранное и список покупок
- **AI-генерации персональных рационов по вкусовым предпочтениям**
- Подписок на авторов
- Микросервисной архитектуры (3 независимых сервиса)

Архитектура системы





Распределение обязанностей

Корекова Снежана: AUTH-SERVICE (Backend Lead)

Роль: Авторизация, пользователи, избранное

Django проект + JWT (Djoser/simplejwt) Модель User (email/username) POST /auth/register, /login, GET /me Модель Favorite (лайки рецептов) POST/GET/DELETE /favorites/{id} Модель Subscription (подписки) POST/GET /users/{id}/subscribe unit-тесты (pytest) Permissions + Throttling Swagger docs Dockerfile

1 этап

Django проект создан + requirements.txt Модель User (extends AbstractUser) POST /auth/register — работает POST /auth/login — выдает JWT токен GET /auth/me — возвращает профиль unit-тесты (pytest) Dockerfile + docker-compose up (auth-db + auth-service) README с инструкцией запуска

2 этап

Добавлена модель Favorite (user, recipe_id) POST/GET/DELETE /favorites/{id} HTTP клиент для recipes-service (mock пока) Добавлена модель Subscription POST/GET /users/{id}/subscribe новые unit-тесты Permissions (IsAuthenticated) Интеграция в общий docker-compose.yml

3 этап

Rate limiting (UserRateThrottle) CORS для фронтенда Swagger/OpenAPI (drf-spectacular) Интеграционные тесты (с recipes-service) Финальные unit-тесты Nginx конфигурация (gateway) Общий docker-compose.yml

Кузнецова Людмила: RECIPES-SERVICE (Core Backend)

Роль: Рецепты, ингредиенты, теги, список покупок
Django проект + DRF Модели: Tag, Ingredient, Recipe, RecipeIngredient GET/POST /recipes (CRUD)
GET /tags, /ingredients (поиск) Фильтры: ?tags=завтрак&is_favorited=1 Список покупок: POST
/shopping_cart/{id} GET /shopping_cart/download (PDF) ImageField (фото рецептов) unit-тесты
Permissions (IsAuthorOrReadOnly) Пагинация + Ordering Dockerfile

1 этап

Django проект + requirements.txt Модели: Tag, Ingredient, Recipe (без RecipeIngredient) GET /tags —
список тегов GET /ingredients — список ингредиентов GET /recipes — пустой список 6 unit-тестов
Dockerfile + docker-compose up (recipes-db + recipes-service) README

2 этап

Модель RecipeIngredient POST /recipes — создание с ингредиентами GET /recipes/{id} —
детальная с вложенными данными PATCH/DELETE /recipes/{id} Фильтры: ?tags=1&is_favorited=1
Модель ShoppingCart POST /shopping_cart/{id}, GET /shopping_cart GET /shopping_cart/download
(PDF) 5 новых unit-тестов (всего 21) ImageField (тестовые фото) Интеграция в docker-compose

3 этап

Permissions (IsAuthorOrReadOnly) Пагинация + Ordering (?ordering=-pub_date) Оптимизация
запросов (select_related/prefetch_related) Изображения (MEDIA_URL + тесты загрузки)
Интеграционные тесты (с auth-service) Финальные unit-тесты

Кондрашов Роман: MEAL-PLANNER-SERVICE (AI Engineer)

Роль: Генерация рационов, AI рекомендации
Django проект + Celery + Redis Модели: UserProfile, MealPlan POST /profiles (предпочтения) POST
/plans/generate (рацион на неделю) Алгоритм similarity_score (теги+ингредиенты) Perplexity API
клиент GET /plans/{id}/suggestions (AI) Redis кеш (1 час) Celery task для async AI HTTP клиент
(recipes-service) 20+ unit-тестов (mock API) Dockerfile

1 этап

Django проект + requirements.txt (celery/redis) Модель UserProfile (user_id, daily_calories) POST
/profiles — сохранить настройки GET /profiles/me — прочитать 4 unit-теста Dockerfile + docker-
compose up (planner-db + planner-service) README

2 этап

HTTP клиент для recipes-service и auth-service Функция calculate_similarity(recipe1, recipe2) POST /plans/generate — рацион на неделю Проверка калорийности и исключений GET /plans/{id} — детальный план новые unit-тесты Интеграция в docker-compose

3 этап

Perplexity API клиент + промпты Celery task для async AI рекомендаций GET /plans/{id}/suggestions (из Redis) Redis кеш (meal_plan:user123) Алгоритм разнообразия (diversity_score) Интеграционные тесты (end-to-end) Postman коллекция (10 AI запросов) Финальные unit-тесты (всего 25+) README с алгоритмами

🔧 Технологический стек

Backend

- **Python 3.11+**
- **Django 5.0+**
- **Django REST Framework 3.14+**
- **djangorestframework-simplejwt** (JWT токены)
- **Djoser** (регистрация/авторизация)
- **drf-spectacular** (OpenAPI/Swagger)
- **Celery 5.3+** (async tasks)
- **requests** (HTTP клиент для межсервисного общения)

Базы данных

- **PostgreSQL 16** (3 отдельные БД для сервисов)
- **Redis 7** (кеш + Celery broker)

DevOps

- **Docker 24+**
- **docker-compose 2.20+**
- **Nginx** (API Gateway)

Тестирование

- **pytest 7.4+**
- **pytest-django**
- **factory-boy** (фикстуры)
- **coverage.py** (code coverage)

Code Quality

- **flake8** (линтер)
- **black** (форматтер)
- **pre-commit** (git hooks)

External APIs

- **Perplexity API** (AI рекомендации)